P3 GC Design Note: Lisp-2 Sliding Mark-Compact (Rust Runtime)

Neha Reddy Sarthak Dayal Jay Shim Rahik Sikder
CS 378H, Fall 2026

1 Choice and Heap Layout

We implemented the *Lisp-2 sliding mark-compact* collector in the Rust skeleton. We chose it because it uses the whole heap rather than half, and because its difficulty sits in one place, the pointer-fixup discipline, which we could enumerate and test directly. The reference Cheney collector stays in `rust/reference/` behind the `reference-cheney` Cargo feature, so we can diff against it.

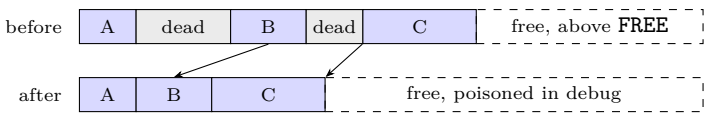
- **Heap.** One contiguous region with a bump pointer `FREE`, 16 MiB by default and overridden by `LO_HEAP_SIZE` at `lo_runtime_init`. Objects sit end to end, so the allocated prefix [base, FREE) can be walked linearly.
- **Allocation.** `lo_alloc` bumps `FREE`. If the object does not fit, it forces one collection and retries once, aborting through `lo_alloc_oom` (exit 137) only if the live set still leaves no room.
- **Sizes and offsets.** All sizes come from `heap_layout::object_size` (`align(instance_size)`, or `align(string_data_offset + length)` for a `String`), and every field offset comes from `offset_of` or the descriptors. Nothing is hard-coded to the 16-byte native or 12-byte WASM header.
- **Mark bit.** Bit 0 of `gc_bits`, clear between collections.
- **Side metadata.** Descriptors must stay intact throughout, since the heap walk reads them, so forwarding addresses are not stored in headers. `compaction::Scratch` instead holds a mark `worklist` and a table of `Relocation{old_offset, new_offset, size}`, both reserved *at init* and never during a collection.
- **No allocation during collection.** A collection never allocates, and re-entry is rejected by an `IN_COLLECTION` assertion.
- **Barrier.** `lo_gc_write_barrier` stays the skeleton's bare store. With one generation there is no old-to-young pointer to miss, so no remembered set or card table is needed.

2 Roots and Collection Passes

The only roots are the shadow-stack slots. The collector walks the `ShadowFrame` chain from `current_frame` and visits each frame's `num_roots` slots, including the static `lo_bindings` frame. Since P2's `gc_roots` spills every live `Ref` around each call, no register or native-stack scan is needed. `managed(p)` is true iff p is non-null and base $\le p <$ FREE, and every visited pointer passes through it. `LO_EMPTY_STRING`, literals, and descriptors live in static data, so they are never marked, moved, or rewritten, with no special case. The ABI supplies object-start pointers only, so an exact-start lookup suffices.

Phase	What it does
`mark`	Marks from the roots with a worklist of heap offsets, so the traversal depth lives in the worklist rather than on the call stack and long link chains cannot overflow the stack. The mark bit is set before pushing, so cycles and aliases enter once. Fields come from the descriptor's `pointer_offsets`, which need not be contiguous.
`compute_forwarding`	Walks the heap with `object_size`. Each marked object gets `new_offset` equal to the live bytes before it. Entries are appended in address order, so the table is sorted by construction.
`fix_references`	Rewrites every root slot and every pointer field of every live object, looking the target object up by binary search. All fixups run while every object is still at its old address.
`slide`	Copies objects down in ascending order with the overlap-safe `ptr::copy`, clearing mark bits. Since new $\le$ old for every object, no unmoved object is overwritten.

Afterward `FREE` is set to the end of the live prefix. With `LO_GC_POISON=1` the freed tail is filled with `0xEF`, so a stale pointer fails loudly.



Fixup argument. After a collection no pointer may hold an old address, so we list every place a managed pointer can live and how each is handled.

1. **Shadow-stack slots.** These are the roots, and `fix_references` rewrites each to its object's new address.
2. **Pointer fields of live objects.** Rewritten for *every* live object, including ones that do not move, because an unmoved object can point at one that does.
3. **Pointers outside the heap.** Descriptors, literals, and `LO_EMPTY_STRING` fail the `managed()` range check and are left alone. They never move, so their addresses are already correct.
4. **Pointers held by the runtime itself.** There are none. String operations copy their inputs into a Rust `Vec` *before* allocating, so no managed pointer is held across a possible collection. `lo_cast_check` and `lo_instanceof` allocate nothing. The worklist and relocation table hold offsets, not pointers.
5. **The bump pointer.** `FREE` is reset to the end of the live prefix.

If these cover every location, nothing holds a stale address after `slide`. Dead objects are never visited, so their stale fields are never read.

String and type operations. The five `lo_string_*` operations are UTF-8 aware (codepoint-aware reversal, byte-wise comparison, exit 120 on a negative repeat count) and follow the root discipline above. `lo_cast_check` and `lo_instanceof` walk the descriptor `parent` chain, with null short-circuits and exit 101 naming both classes.

3 Testing on Both Targets

LO stress tests (suite header-contract format) are in `tests/gc-stress/`. There are five, and each allocates about 3 million objects, keeps live data under a quarter of its heap, and was confirmed to fail on a copy of our runtime with the named bug planted.

Test	Invariant (planted bug it catches)
`gc_01_unmoved_anchor_fixup`	Fixup rewrites fields of unmoved live objects (fixup skips objects whose address is unchanged)
`gc_02_overlapping_slide`	A slide shorter than the object is overlap-safe (copying high to low, or the tail block first)
`gc_03_survivors_then_die`	Survivors are reclaimed once dead (mark bits left set)
`gc_04_deep_list_marking`	Marking uses a worklist (recursive marking overflows the stack on a very long link chain)
`gc_05_every_pointer_field`	Scan visits exactly the descriptor's offsets (first field only, last field skipped, contiguity assumed)

`run.sh` runs each program on WASM and native at the default heap, and on native at its declared heap. All five pass on both targets, and natively at their declared heaps. A WASM module has no environment, so `LO_HEAP_SIZE` cannot reach it and WASM always runs at 16 MiB. There, collections come from allocating past that heap.

Forced collections. LO source has no way to call `lo_gc_collect`, which the ABI reserves for tests, so these programs force collections by exhausting the heap instead. Each allocates about 3 million short-lived objects against its declared heap, so collections recur throughout the run while the program's live data is in a known shape. Direct `lo_gc_collect` calls are exercised by Rust tests, run natively and compiled to WASM, which `tests/gc-stress/README.md` maps to each stress-test category.

4 Challenges and Limitations

Tradeoffs.

- **Lisp-2 over Cheney.** We use the whole heap instead of half, but make three passes per collection instead of one copy.
- **Side tables over header forwarding.** Descriptors stay intact for the heap walk, but fixup needs a binary search per pointer rather than a direct read.
- **Scratch reserved at init.** A collection never allocates, but the tables reserve about twice the heap in address space.
- **Non-generational.** The barrier stays a bare store and nothing can be missed, but every collection scans all live data.

Limitations.

- There is no heap-checker walk and no per-collection GC log, only poisoning.
- WASM always runs at the default 16 MiB heap, so small heaps are tested natively only.
- Out-of-memory inside `lo_string_repeat` still exits 120 on WASM, since `wasmrun` matches that name first.

Exit-code hardening. We ran programs that exhaust the heap inside a string operation on both targets: `lo_string_concat` exited 137 natively but 134 on WASM. The course's `wasmrun` chooses a trap's exit code by searching its backtrace for function names and reports 137 only when a frame named `lo_alloc*` is present. String operations allocate through the variable-size allocator rather than `lo_alloc`, so no frame on that path matched. Every out-of-memory abort now goes through one function, `lo_alloc_oom`, marked `#[inline(never)]` so it always appears in the backtrace.